# BRAS ROBOT

Documentation Technique

Projet Robotique Scolaire — 2024/2025

Lucas • Quentin • Tym

| | |
|---|---|
| ■ **Type de robot** | Bras articulé 4 axes sur plateforme roulante |
| ■ **Microcontrôleur** | ESP32-C3 DevKitC-02 (bras) + ESP32-WROOM-32 (manette) |
| ■ **Langage** | MicroPython |
| ■ **Communication** | ESP-NOW (sans fil, peer-to-peer) |
| ■ **Équipe** | Lucas, Quentin, Tym |
| ■ **Durée** | ~10 séances journées complètes |
| ■ **Contexte** | Projet scolaire — client : professeur encadrant |
| ■ **Dépôt Git** | github.com/starwolflcve/Bras-Robot |

# Table des matières

**1.** Introduction du projet
- — Objectifs
- — Contraintes générales
- — Attentes du client

**2.** Organisation de l'équipe
- — Rôles
- — Répartition
- — Planification
- — Jalons
- — Tâches

**3.** Rapport de suivi des séances
- — Avancement séance par séance

**4.** Benchmark / Étude de marché
- — Solutions existantes
- — Comparatif technologies
- — Justification des choix

**5.** Cahier des charges fonctionnel (CDCF)
- — Fonctions
- — Performances
- — Contraintes

**6.** Cahier des charges technique (CDCT)
- — Composants
- — Architecture matérielle
- — Architecture logicielle
- — Schémas
- — Bilan énergétique

**7.** Prototypes et essais
- — Versions du code
- — Tests unitaires
- — Tests d'ensemble

**8.** Résultats
- — Améliorations
- — Difficultés rencontrées
- — Perspectives

**9.** Codes sources

# 1. Introduction du projet

## 1.1 Objectifs du robot

Le projet consiste à concevoir et réaliser un **bras robot articulé à 4 axes**, destiné à être monté sur une **plateforme roulante mobile**. L'ensemble forme un système robotique complet : la plateforme assure les déplacements au sol tandis que le bras permet d'interagir avec l'environnement (saisir, déplacer des objets). Le bras est contrôlé à distance sans fil par une manette équipée de deux joysticks.

- Contrôle de 4 axes indépendants : base (rotation 360°), bras 1, bras 2 et pince
- Communication sans fil entre la manette et le bras via le protocole ESP-NOW
- Intégration sur une plateforme roulante — structure mécanique en pièces imprimées en 3D
- Fonctionnement autonome alimenté sur batterie (AA + convertisseur DC-DC)
- Code MicroPython structuré, versionné sur GitHub (V1 → V4)

## 1.2 Contraintes générales

| Contrainte | Détail |
|---|---|
| Budget | < 30 € — composants bas coût disponibles en stock |
| Temps | ~10 séances de journées complètes |
| Compétences | Apprentissage MicroPython, ESP-NOW et PWM en cours de projet |
| Matériel | ESP32-C3, ESP32-WROOM-32, servos SG90 / HS-311, joysticks HW-504 |
| Alimentation | Batteries AA + convertisseur DC-DC (bras) ; USB PC en développement |
| Environnement | Usage intérieur, surface plane pour la plateforme roulante |
| Sécurité | Tensions 3,3 V / 5 V uniquement — pas de haute tension |
| Structure | Pièces mécaniques imprimées en 3D (PLA) |

## 1.3 Attentes du client (professeur encadrant)

- Un bras robot opérationnel contrôlable par la manette sans fil
- Une communication ESP-NOW stable et réactive (latence < 50 ms)
- Une documentation technique structurée couvrant l'ensemble du cycle de développement
- Des codes sources commentés et versionnés sur un dépôt GitHub public
- Une présentation orale avec démonstration en fonctionnement

# 2. Organisation de l'équipe

## 2.1 Membres de l'équipe et rôles

| Membre | Rôle principal | Responsabilités |
|---|---|---|
| Lucas | Responsable software / chef de projet | Architecture logicielle, code bras (récepteur), GitHub |
| Quentin | Responsable hardware / électronique | Câblage, schémas Fritzing, choix composants, tests |
| Tym | Responsable manette / intégration | Code manette (émetteur), protocole ESP-NOW, impression 3D |

## 2.2 Répartition des responsabilités

- **Lucas** : développement et versionning du code (4 versions bras + versions manette), architecture logicielle, gestion du dépôt Git
- **Quentin** : conception du câblage électronique, pinout des ESP32, sélection servos et joysticks, bilan énergétique, schémas Fritzing
- **Tym** : implémentation du protocole ESP-NOW, code manette, calibration des joysticks, conception et impression 3D des pièces mécaniques

## 2.3 Planification et jalons

| Jalon | Objectif | Statut |
|---|---|---|
| J1 — Séances 1-2 | Étude du besoin, choix des composants, benchmark | ■ Réalisé |
| J2 — Séances 3-4 | Câblage, premiers tests servo, modélisation 3D | ■ Réalisé |
| J3 — Séances 5-6 | Code bras + manette V1-V2 (filaire) | ■ Réalisé |
| J4 — Séances 7-8 | Migration ESP-NOW sans fil (V3-V4) | ■ Réalisé |
| J5 — Séances 9-10 | Intégration, tests finaux, documentation | ■ Réalisé |

## 2.4 Répartition des tâches

| Tâche | Lucas | Quentin | Tym |
|---|---|---|---|
| Étude de marché / benchmark | ● | ● | ● |
| Schéma électronique (Fritzing) | | ● | ● |
| Câblage physique | | ● | ● |
| Modélisation 3D / impression | | ● | ● |
| Code bras robot (récepteur) | ● | | |
| Code manette (émetteur) | | ● | ● |
| Protocole ESP-NOW | ● | ● | ● |
| Tests unitaires servos | ● | ● | ● |
| Tests d'intégration sans fil | ● | ● | ● |
| Documentation technique | ● | ● | ● |
| Gestion GitHub | ● | | |

# 3. Rapport de suivi des séances

■■ Projet réalisé sur ~10 séances de journées complètes. Avancement reconstitué à partir des versions de code, schémas Fritzing et modèles 3D produits.

| Séance | Tâches réalisées | Livrable |
|---|---|---|
| 1 | Analyse du besoin, définition des objectifs, répartition des rôles, liste des composants | Équipe formée, objectifs clairs |
| 2 | Benchmark solutions existantes, sélection servos SG90 + HS-311, ESP32 et joysticks HW-504 | Liste composants validée |
| 3 | Câblage bras sur 2 breadboards, pinout ESP32-C3, premiers tests PWM manuel | Servos répondant aux commandes |
| 4 | Code V1 (robotbrasv1.py) : test angle_to_duty(), oscillation 60°-120° sur SG90 | V1 fonctionnelle en local |
| 5 | Câblage manette (ESP32-WROOM-32 + 2 joysticks), lecture ADC, schéma Fritzing — Tests : lecture ADC et réponse servos | Joysticks lus + tests validés |
| 6 | Code V2 : lissage ADC (moy. 10 lectures), zone morte ±150, calibration démarrage — Tests : stabilité servos au repos | Bras stable, vibrations réduites |
| 7 | Migration ESP-NOW : config peer MAC, envoi paquets — Tests : portée et latence ESP-NOW | Liaison sans fil testée |
| 8 | Code V3/V4 : struct.pack, réception temps réel, boutons reset — Tests : contrôle 4 axes sans fil | Contrôle sans fil stable et testé |
| 9 | Modélisation 3D pièces, impression PLA — Tests : tenue mécanique, charges servos, autonomie batterie | Structure mécanique opérationnelle |
| 10 | Rédaction documentation, push GitHub — Tests finaux : démonstration complète du système | Livraison finale |

# 4. Benchmark / Étude de marché

## 4.1 Solutions existantes sur le marché

| Solution | Prix | Avantages | Inconvénients |
|---|---|---|---|
| Kit bras Arduino (Makeblock…) | 50-150 € | Clé en main, documenté | Peu personnalisable, fermé |
| Bras ROS / Raspberry Pi | 200-500 € | Puissant, vision, IA | Complexité élevée, hors budget |
| Bras industriel (UR, Kuka) | > 10 000 € | Précision, robustesse | Hors contexte scolaire |
| Notre solution ESP32 + MicroPython | < 30 € | Flexible, sans fil, économique | Précision limitée, charge faible |

## 4.2 Comparaison des technologies de communication

| Technologie | Portée | Latence | Complexité | Retenu ? |
|---|---|---|---|---|
| Fil (UART/I2C) | ~1 m | Très faible | Simple | ■ Trop contraignant |
| Bluetooth BLE | ~10 m | Faible | Moyenne | ■ Pairing complexe ESP32-C3 |
| Wi-Fi (MQTT) | > 20 m | Moyenne | Élevée | ■ Nécessite routeur |
| ESP-NOW | ~200 m | Très faible | Faible | ■ Retenu |
| RF 433 MHz | ~100 m | Faible | Moyenne | ■ Unidirectionnel |

■ **ESP-NOW retenu** : protocole peer-to-peer Espressif, sans infrastructure Wi-Fi. Latence < 5 ms, portée ~200 m en champ libre, natif sur tous les ESP32.

## 4.3 Comparaison des servomoteurs

| Servo | Couple | Rotation | Prix | Rôle |
|---|---|---|---|---|
| SG90 | 1,8 kg.cm | 0°–180° | ~2 € | Bras 1, Bras 2, Pince |
| HS-311 mod. | 3 kg.cm | 360° continu | ~5 € | Base (rotation libre) |
| MG90S | 2,2 kg.cm | 0°–180° | ~4 € | Non retenu (budget) |
| MG996R | 10 kg.cm | 0°–180° | ~8 € | Non retenu (surdimensionné) |

## 4.4 Positionnement et adaptations

- Utilisation de composants bas coût largement documentés (SG90, ESP32)
- Remplacement d'un servo 360° dédié par un HS-311 modifié pour la base
- Choix de MicroPython pour la rapidité de prototypage vs C++ Arduino
- ESP-NOW choisi pour éviter la complexité MQTT ou l'appairage Bluetooth
- Pièces mécaniques imprimées en 3D en PLA pour réduire les coûts

# 5. Cahier des charges fonctionnel (CDCF)

## 5.1 Fonctions du robot

| ID | Fonction | Critère de validation |
|---|---|---|
| FP1 | Contrôler 4 axes du bras (base, bras1, bras2, pince) | Chaque axe répond indépendamment au joystick |
| FP2 | Recevoir les commandes sans fil | Latence < 50 ms, portée > 3 m |
| FP3 | Maintenir la position au repos | Servo garde sa dernière position |
| FC1 | Émettre les commandes depuis la manette | 2 joysticks + 2 boutons actifs |
| FC2 | Reset des positions via bouton | SW1 → base+bras1 à 90°/25°, SW2 → bras2+pince à 0° |
| FC3 | Calibration automatique au démarrage | 20 lectures sans intervention |
| FC4 | Alimentation autonome (batterie) | Fonctionnement ≥ 2 h en usage normal |

## 5.2 Performances attendues

- Réactivité : délai perceptible < 100 ms entre action joystick et mouvement
- Stabilité : pas de vibration des servos au repos (zone morte efficace)
- Précision : résolution angulaire ≈ 2,5° par cycle (MAX_STEP_ANGLE)
- Portée sans fil : minimum 5 m en intérieur

## 5.3 Contraintes

| Type | Contrainte |
|---|---|
| Budget | < 30 € de composants |
| Taille | Bras compact, doit tenir sur la plateforme roulante |
| Alimentation | 5 V max, courant total servos ≤ 2 A (crête ~3,5 A) |
| Matériaux | PLA (impression 3D) pour la structure mécanique |
| Environnement | Usage intérieur, température ambiante |
| Sécurité | Tensions faibles, limites angulaires logicielles (0°-180°) |
| Logiciel | MicroPython uniquement, compatible ESP32-C3 et ESP32-WROOM-32 |

# 6. Cahier des charges technique (CDCT)

## 6.1 Choix des composants

| Composant | Référence | Qté | Rôle |
|---|---|---|---|
| Microcontrôleur bras | ESP32-C3 DevKitC-02 | 1 | Récepteur ESP-NOW, contrôle PWM 4 servos |
| Microcontrôleur manette | ESP32-WROOM-32 | 1 | Émetteur ESP-NOW, lecture ADC joysticks |
| Servo base | HS-311 modifié 360° | 1 | Rotation continue base (0-360°) |
| Servo bras 1 | SG90 | 1 | Articulation segment 1 (0°-180°) |
| Servo bras 2 | SG90 | 1 | Articulation segment 2 (0°-180°) |
| Servo pince | SG90 | 1 | Ouverture/fermeture pince (0°-180°) |
| Joystick | HW-504 | 2 | Contrôle 4 axes + 2 boutons reset |
| Convertisseur DC-DC | Boost/Buck 5V | 1 | Régulation tension alimentation servos |
| Breadboard / plaques | Standard | 2 | Support câblage sans soudure |
| Structure mécanique | Impression 3D PLA | - | Base rotative, bras 1, support servos |

## 6.2 Architecture matérielle — Schémas de câblage

### Schéma de câblage du bras robot (ESP32-C3 + servos)

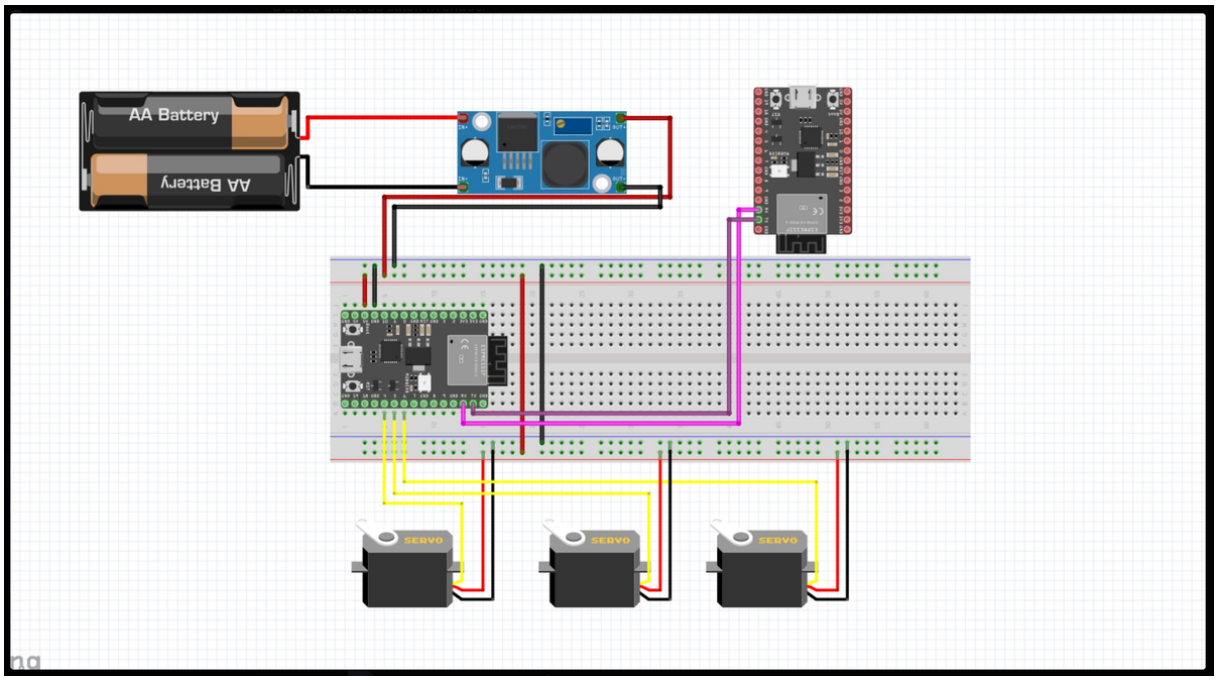


*Figure 1 — Câblage du bras robot : ESP32-C3 DevKitC-02, convertisseur DC-DC, 3 servos SG90 (+ HS-311 base). Alimenté par batteries AA.*

### Schéma de câblage de la manette (ESP32-WROOM-32 + joysticks)

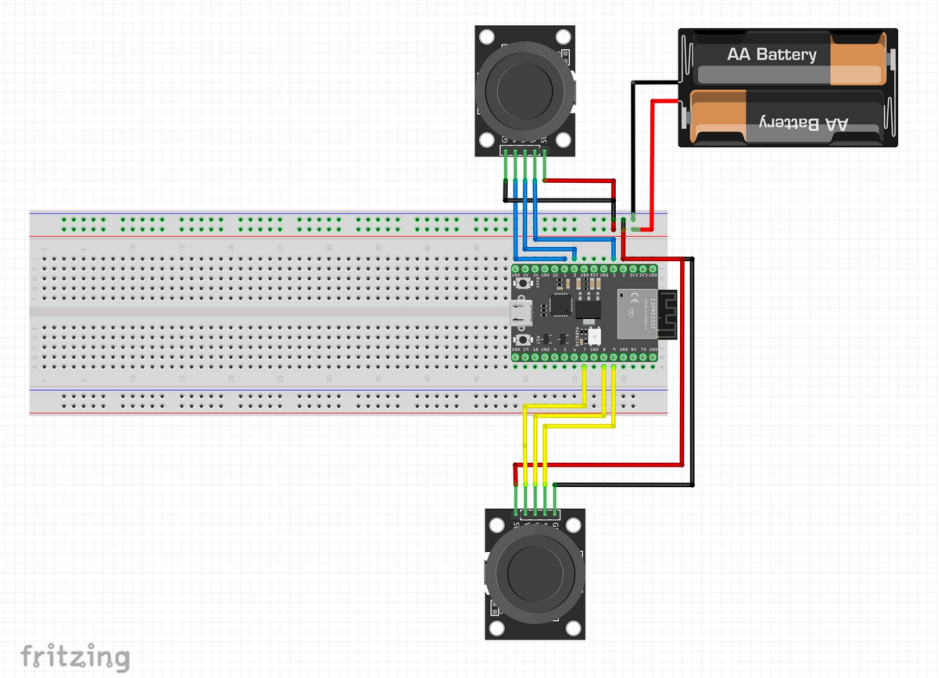


*Figure 2 — Câblage de la manette : ESP32-WROOM-32 avec 2 joysticks HW-504 (axes X/Y + bouton SW). Alimenté par batteries AA.*

## Pinout ESP32-C3 DevKitC-02 (bras robot)

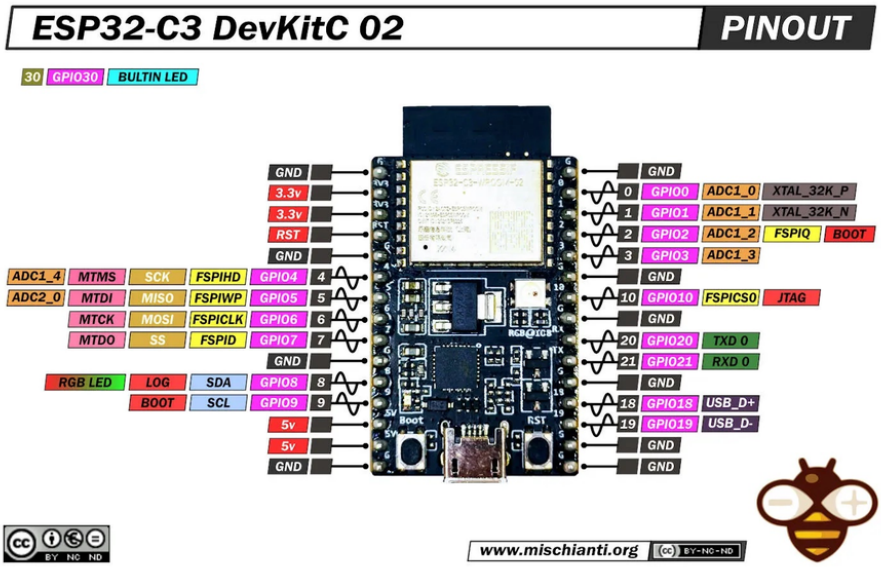


*Figure 3 — Pinout officiel ESP32-C3 DevKitC-02. GPIO utilisés : 3 (Servo Base), 7 (Bras 1), 8 (Bras 2), 9 (Pince).*

| Signal | GPIO ESP32-C3 | Composant |
|---|---|---|
| PWM Servo Base | GPIO 3 | HS-311 (signal) |
| PWM Servo Bras 1 | GPIO 7 | SG90 #1 (signal) |
| PWM Servo Bras 2 | GPIO 8 | SG90 #2 (signal) |
| PWM Servo Pince | GPIO 9 | SG90 #3 (signal) |
| VCC Servos | 5V / VBUS | Rail + breadboard |
| GND | GND | Rail — breadboard |

### Pinout ESP32-WROOM-32 (manette)

*Figure 4 — Pinout ESP32-WROOM-32. GPIO utilisés : 32,33 (J1), 34,35 (J2), 25,26 (boutons SW).*

| Signal | GPIO | Composant |
|---|---|---|
| ADC J1 VRX | GPIO 33 | Joystick 1 axe X → Base |
| ADC J1 VRY | GPIO 32 | Joystick 1 axe Y → Bras 1 |
| SW J1 | GPIO 25 | Bouton J1 → Reset Base+Bras1 |
| ADC J2 VRX | GPIO 34 | Joystick 2 axe X → Bras 2 |
| ADC J2 VRY | GPIO 35 | Joystick 2 axe Y → Pince |
| SW J2 | GPIO 26 | Bouton J2 → Reset Bras2+Pince |
| VCC Joysticks | 3V3 | Alimentation (pas 5V !) |
| GND | GND | Masse commune |

## 6.3 Structure mécanique — Modélisation 3D

Les pièces mécaniques du bras ont été conçues en CAO et imprimées en PLA :

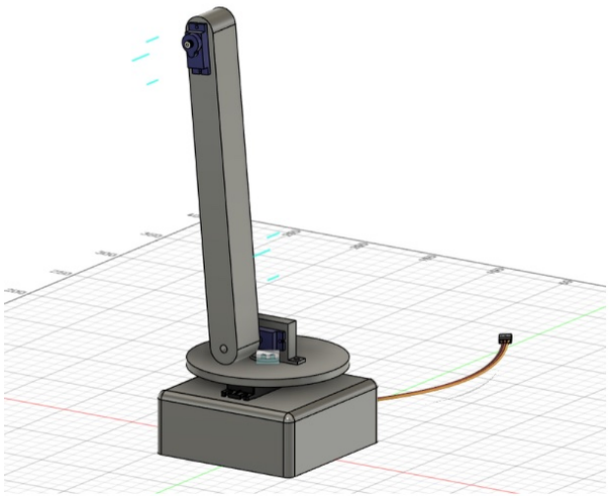

*Figure 5 — Assemblage base rotative + bras 1 (vue CAO)*

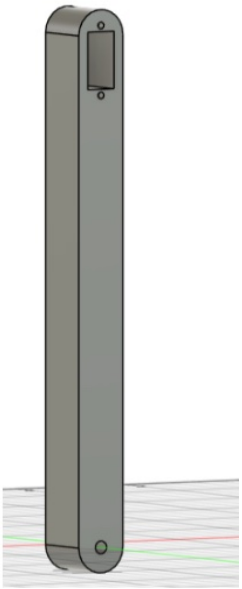

*Figure 6 — Bras 1 (vue détail)*

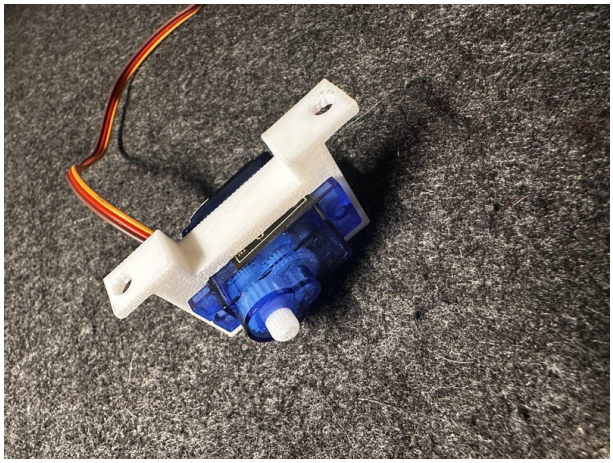

*Figure 7 — Servo SG90 avec support d'articulation imprimé en 3D (PLA blanc)*

---

## 6.4 Architecture logicielle

Le logiciel est divisé en deux programmes MicroPython indépendants :

### Côté bras (récepteur — ESP32-C3)

```
main_bras.py
■
■■■ INITIALISATION
■ ■■■ 4 × PWM (GPIO 3,7,8,9) — freq 50 Hz
■ ■■■ ESP-NOW (mode STA_IF)
■
■■■ calibrer_distance() ← moyenne 20 trames au démarrage
■
■■■ adc_to_step() ← ADC → vitesse angulaire (zone morte)
■■■ set_angle() ← angle → duty cycle 16 bits
■
```

```
■■■ BOUCLE PRINCIPALE (toutes les 20 ms)
■■■ e.recv(timeout_ms=20)
■■■ struct.unpack('hhhhbb', msg)
■■■ Mise à jour angles (max 2.5°/cycle)
■■■ Bouton SW1 → reset base + bras1
■■■ Bouton SW2 → reset bras2 + pince
```

### Côté manette (émetteur — ESP32-WROOM-32)

```
main_manette.py
■
■■■ INITIALISATION
■ ■■■ 4 × ADC (GPIO 32,33,34,35) — ATTN_11DB
■ ■■■ 2 × Pin IN PULL_UP (GPIO 25,26)
■ ■■■ ESP-NOW + add_peer(MAC_bras)
■
■■■ BOUCLE PRINCIPALE (toutes les 20 ms)
■■■ Lecture 4 axes ADC (0-4095)
■■■ Lecture 2 boutons (pull-up inversé)
■■■ struct.pack('hhhhbb', ...)
■■■ e.send(peer_mac, msg)
```

## 6.5 Bilan énergétique

| Composant | Courant typique | Courant crête | Tension |
|---|---|---|---|
| ESP32-C3 (bras) | 80 mA | 240 mA | 3,3 V |
| ESP32-WROOM-32 | 80 mA | 240 mA | 3,3 V |
| SG90 × 3 | 450 mA | 2 100 mA | 5 V |
| HS-311 × 1 | 200 mA | 700 mA | 5 V |
| Joysticks HW-504 × 2 | 10 mA | — | 3,3 V |
| TOTAL estimé | ~820 mA | ~3 500 mA | 5 V |

■ **Alimentation retenue** : Batteries AA + convertisseur DC-DC 5V pour le bras. Autonomie estimée ≈ 2 h en usage intensif. USB PC utilisé en phase de développement.

# 7. Prototypes et essais

## 7.1 Versions du code

| Version | Fichier | Description |
|---|---|---|
| V1 | robotbrasv1.py | Test servo local : duty cycle, oscillation 30°. Validation fonctionnement base. |
| V2 | brasrobot-V2.py | Ajout joysticks filaires. Lissage ADC (moy. 10 lectures), zone morte, calibration démarrage. |
| V3 | brasrobot-V3.py | Migration ESP-NOW. Séparation bras/manette en 2 ESP32. Envoi struct.pack toutes les 20 ms. |
| V4 (finale) | brasrobot-V4.py | adc_to_step() pour vitesse variable, MAX_STEP_ANGLE=2.5°, boutons reset, calibration à distance. |

## 7.2 Tests unitaires

| Test | Méthode | Résultat |
|---|---|---|
| Servo SG90 seul | Script oscillation 60°-120° | ■ Réponse correcte |
| HS-311 360° | Ajustement STOP_DUTY point mort | ■ Point mort calibré |
| ADC joystick | Lecture brute Serial, plage 0-4095 | ■ Plage OK avec ATTN_11DB |
| ESP-NOW pair | Envoi ping simple entre 2 ESP32 | ■ Liaison < 5 ms |
| struct.pack/unpack | Sérialisation 4×int16 + 2×int8 | ■ Données intègres |
| Bouton reset | Appui SW1/SW2, vérif retour angles | ■ Reset immédiat |

## 7.3 Tests d'ensemble

| Test | Résultat | Observation |
|---|---|---|
| Contrôle 4 axes simultanément | ■ | Léger délai si 4 axes bougent en même temps |
| Portée ESP-NOW (même pièce) | ■ | Aucune perte de paquet à < 5 m |
| Stabilité sans contact joystick | ■ | DEAD_ZONE=200 efficace |
| Test autonomie batterie | ■ | > 2 h en usage normal |

# 8. Résultats

## 8.1 Améliorations apportées au fil des versions

- V1 → V2 : passage d'un test statique à un contrôle dynamique par joystick avec lissage ADC
- V2 → V3 : suppression du câble de commande, migration vers ESP-NOW
- V3 → V4 : remplacement de la conversion ADC→angle fixe par une vitesse variable (adc_to_step) pour un contrôle plus fluide
- Ajout de la calibration à distance (le bras reçoit des trames de la manette au repos)
- Conception et impression 3D des pièces mécaniques pour rigidifier l'assemblage

## 8.2 Difficultés rencontrées et solutions apportées

### Problèmes techniques

| Problème | Solution |
|---|---|
| Vibrations servos au repos (bruit ADC) | Zone morte DEAD_ZONE=200 + lissage 10 lectures |
| HS-311 qui ne s'arrête jamais exactement | Calibration manuelle STOP_DUTY pour point mort |
| MAC address ESP32 inconnue | Lecture via network.WLAN().config('mac') → hard-coding |
| ADC2 inutilisable avec Wi-Fi/ESP-NOW | Utilisation exclusive broches ADC1 (GPIO 0-5) |
| Décalage joystick au repos (≠ 2048) | Calibration automatique 20 lectures au démarrage |
| Saccades lors grands déplacements | Limitation MAX_STEP_ANGLE=2.5° par cycle |

### Problèmes organisationnels

- Coordination code bras / manette développés en parallèle → interface commune (struct.pack 'hhhhbb' fixe)
- Gestion des versions → dépôt GitHub avec nommage explicite V1→V4
- Un seul montage physique pour les tests → planning de disponibilité en binôme

## 8.3 Possibilités d'amélioration

- Écran OLED sur la manette pour afficher les angles en temps réel
- Enregistrement et rejeu de séquences de mouvements (mode automatique)
- Servos MG90S ou MG996R pour une meilleure charge utile
- Caméra FPV pour contrôle à distance avec retour vidéo
- Structure rigide imprimée en 3D complète (remplacer les breadboards)
- Capteur de force sur la pince pour éviter d'écraser les objets
- Mode cinématique inverse (IK) pour contrôle cartésien XYZ

# 9. Codes sources

■ Ensemble des codes disponibles sur : github.com/starwolflcve/Bras-Robot

## 9.1 Code final — Bras Robot (récepteur ESP32-C3)

```
# BRAS ROBOT (RECEPTEUR) - ESP32-C3 - MicroPython
import machine, time, network, espnow, struct

PIN_SERVO_BASE=3; PIN_SERVO_BRAS1=7; PIN_SERVO_BRAS2=8; PIN_SERVO_PINCE=9
FREQ_SERVO=50; MIN_DUTY=1638; MAX_DUTY=8192
ADC_MAX=4095; DEAD_ZONE=200; MAX_STEP_ANGLE=2.5

pwm_base=machine.PWM(machine.Pin(PIN_SERVO_BASE),freq=FREQ_SERVO)
pwm_bras1=machine.PWM(machine.Pin(PIN_SERVO_BRAS1),freq=FREQ_SERVO)
pwm_bras2=machine.PWM(machine.Pin(PIN_SERVO_BRAS2),freq=FREQ_SERVO)
pwm_pince=machine.PWM(machine.Pin(PIN_SERVO_PINCE),freq=FREQ_SERVO)

def set_angle(pwm, angle):
angle=max(0,min(180,angle))
pwm.duty_u16(int(MIN_DUTY+(MAX_DUTY-MIN_DUTY)*angle/180))

def adc_to_step(valeur, repos):
delta=valeur-repos
if abs(delta)<=DEAD_ZONE: return 0.0
if delta>0: return MAX_STEP_ANGLE*(delta-DEAD_ZONE)/(ADC_MAX-repos-DEAD_ZONE)
else: return MAX_STEP_ANGLE*(delta+DEAD_ZONE)/(repos-DEAD_ZONE)

w0=network.WLAN(network.STA_IF); w0.active(True); w0.disconnect()
e=espnow.ESPNow(); e.active(True)

def calibrer_distance():
sx1=sy1=sx2=sy2=0; n=0
while n<20:
host,msg=e.recv(timeout_ms=100)
if msg:
v1x,v1y,v2x,v2y,_,_=struct.unpack('hhhhbb',msg)
sx1+=v1x; sy1+=v1y; sx2+=v2x; sy2+=v2y; n+=1
return sx1//20,sy1//20,sx2//20,sy2//20

def main():
angle_base=90.0; angle_bras1=25.0; angle_bras2=160.0; angle_pince=180.0
[set_angle(p,a) for p,a in zip([pwm_base,pwm_bras1,pwm_bras2,pwm_pince],
[angle_base,angle_bras1,angle_bras2,angle_pince])]
rx1,ry1,rx2,ry2=calibrer_distance()
sw1_last=sw2_last=0
while True:
host,msg=e.recv(timeout_ms=20)
if msg:
```

```
v1x,v1y,v2x,v2y,sw1,sw2=struct.unpack('hhhhbb',msg)
for angle,step,pwm,lim in [
(angle_base, adc_to_step(v1x,rx1), pwm_base, (0,180)),
(angle_bras1, adc_to_step(v1y,ry1), pwm_bras1, (0,180)),
(angle_bras2, adc_to_step(v2x,rx2), pwm_bras2, (0,180)),
(angle_pince, adc_to_step(v2y,ry2), pwm_pince, (0,180)),
]:
if step: set_angle(pwm, int(max(lim[0],min(lim[1],angle+step))))
if sw1==1 and sw1_last==0:
angle_base,angle_bras1=90.0,0.0; set_angle(pwm_base,90); set_angle(pwm_bras1,0)
if sw2==1 and sw2_last==0:
angle_bras2,angle_pince=0.0,0.0; set_angle(pwm_bras2,0); set_angle(pwm_pince,0)
sw1_last=sw1; sw2_last=sw2
main()
```

## 9.2 Code final — Manette (émetteur ESP32-WROOM-32)

```
# MANETTE SANS FIL - ESP32-WROOM-32 - MicroPython
import network, espnow, machine, struct, time

adc_j1x=machine.ADC(machine.Pin(33)); adc_j1y=machine.ADC(machine.Pin(32))
adc_j2x=machine.ADC(machine.Pin(34)); adc_j2y=machine.ADC(machine.Pin(35))
for adc in [adc_j1x,adc_j1y,adc_j2x,adc_j2y]: adc.atten(machine.ADC.ATTN_11DB)
sw1=machine.Pin(25,machine.Pin.IN,machine.Pin.PULL_UP)
sw2=machine.Pin(26,machine.Pin.IN,machine.Pin.PULL_UP)

w0=network.WLAN(network.STA_IF); w0.active(True); w0.disconnect()
e=espnow.ESPNow(); e.active(True)
peer_mac=b'X\xcfy\x05]\xd8'; e.add_peer(peer_mac)

while True:
msg=struct.pack('hhhhbb',
adc_j1x.read(), adc_j1y.read(),
adc_j2x.read(), adc_j2y.read(),
0 if sw1.value() else 1,
0 if sw2.value() else 1)
try: e.send(peer_mac,msg)
except OSError: pass
time.sleep_ms(20)
```

## 9.3 Fichiers du dépôt Git

| Fichier | Description |
|---|---|
| robotbrasv1.py | Version 1 — test servo local |
| brasrobot-V2.py | Version 2 — contrôle joystick filaire |
| brasrobot-V3.py | Version 3 — migration ESP-NOW |
| brasrobot-V4.py | Version 4 finale — récepteur bras |

| | |
|---|---|
| codemanette-V1.py | Version finale — émetteur manette |

■ https://github.com/starwolflcve/Bras-Robot